In [128…
```
import pandas as pd
import numpy as np
```

Obsługa **indeksowania hierarchicznego** jest ważnym elementem biblioteki pandas umożliwiającym przypisanie do jednej osi wielu poziomów indeksowania (przypisanie dwóch lub więcej indeksów).

In [129…
```
data = pd.Series(np.random.uniform(size=9),
                 index=[["a", "a", "a", "b", "b", "c", "c", "d", "d"],
                        [1, 2, 3, 1, 3, 1, 2, 2, 3]])
```

In [130…
```
data
```

Out[130…

		0
a	**1**	0.907566
	2	0.249292
	3	0.410383
b	**1**	0.755551
	3	0.228798
c	**1**	0.076980
	2	0.289751
d	**2**	0.161221
	3	0.929698

dtype: float64

Wyświetloną czytelnie serią, której indeksem jest obiekt MultiIndex.

In [131…
```
data.index
```

Out[131…
```
MultiIndex([('a', 1),
            ('a', 2),
            ('a', 3),
            ('b', 1),
            ('b', 3),
            ('c', 1),
            ('c', 2),
            ('d', 2),
            ('d', 3)],
           )
```

Obiekty o indeksie hierarchicznym obsługują tzw. **indeksowanie częściowe**. Indeksowanie to umożliwia zwięzły wybór podzbioru danych

In [132…
```
data["b"]
```

Out[132…

	0
1	0.755551
3	0.228798

dtype: float64

In [133…

```
data["b":"c"]
```

Out[133…

		0
b	**1**	0.755551
	3	0.228798
c	**1**	0.076980
	2	0.289751

dtype: float64

In [134…

```
data.loc[["b", "d"]]
```

Out[134…

		0
b	**1**	0.755551
	3	0.228798
d	**2**	0.161221
	3	0.929698

dtype: float64

Przykład na danych

In [135…

```
# Dane sprzedażowe w dwóch miastach, w dwóch latach
df = pd.DataFrame({
    "miasto":   ["Łódź", "Łódź", "Kraków", "Kraków"],
    "rok":      [2023,   2024,   2023,     2024],
    "sprzedaz": [120,    145,    200,      230]
})
df
```

Out[135…

	miasto	rok	sprzedaz
0	Łódź	2023	120
1	Łódź	2024	145
2	Kraków	2023	200
3	Kraków	2024	230

In [136…

```
df.index
```

Out[136…

```
RangeIndex(start=0, stop=4, step=1)
```

Zwykły filtrowanie

Za każdym razem: filtr po mieście, filtr po roku, wybór kolumny. Trzy operacje dla jednej wartości.

In [137…

```
# Sprzedaż w łodzi w 2024 - działa, ale rozwlekle:
df[(df.miasto == "Łódź") & (df.rok == 2024)]["sprzedaz"]
```

Out[137…

	sprzedaz
1	145

dtype: int64

In [138…

```
# Sprzedaż w Krakowie w 2023 - znowu to samo:
df[(df.miasto == "Kraków") & (df.rok == 2023)]["sprzedaz"]
```

Out[138…

	sprzedaz
2	200

dtype: int64

MultiIndex - indeks jako "ścieżka"

In [139…

```
# Ten sam zbiór, ale z hierarchicznym indeksem:
sprzedaz = pd.Series(
    [120, 145, 200, 230],
    index=[["Łódź", "Łódź", "Kraków", "Kraków"],
           [2023,   2024,   2023,     2024]]
)
print(sprzedaz)
```

```
Łódź    2023    120
        2024    145
Kraków  2023    200
        2024    230
dtype: int64
```

In [140…
```
# Sam indeks to obiekt MultiIndex:
sprzedaz.index
```

Out[140…
```
MultiIndex([(  'Łódź', 2023),
            (  'Łódź', 2024),
            ('Kraków', 2023),
            ('Kraków', 2024)],
           )
```

Indeksowanie - podstawowe wzorce

In [141…
```
# Pojedyncza wartość - krotka (miasto, rok):
sprzedaz["Łódź", 2024]
```

Out[141…
```
np.int64(145)
```

In [142…
```
# Wszystko dla łodzi (wybór zewnętrznego poziomu):
sprzedaz["Łódź"]
```

Out[142…

	0
2023	120
2024	145

dtype: int64

In [143…
```
# Wszystkie miasta, ale tylko rok 2023 (wewnętrzny poziom):
sprzedaz.loc[:, 2023]
```

Out[143…

	0
Łódź	120
Kraków	200

dtype: int64

In [144…
```
# Zakres miast (uwaga: wymaga posortowanego indeksu):
sprzedaz.sort_index().loc["Kraków":"Łódź"]
```

Out[144…

		0
Kraków	**2023**	200
	2024	230
Łódź	**2023**	120
	2024	145

dtype: int64

Z Series do DataFrame i z powrotem

In [145…
```
print(sprzedaz)
```

```
łódź    2023    120
        2024    145
Kraków  2023    200
        2024    230
dtype: int64
```

In [146…
```
# unstack - "wypchnięcie" wewnętrznego poziomu do kolumn:
tabela = sprzedaz.unstack()
print(tabela)
print(type(tabela))
```

```
        2023  2024
Kraków   200   230
łódź     120   145
<class 'pandas.core.frame.DataFrame'>
```

In [147…
```
# stack - operacja odwrotna: kolumny wracają do indeksu:
tab_stack = tabela.stack()
print(tab_stack)
print(type(tab_stack))
```

```
Kraków  2023    200
        2024    230
łódź    2023    120
        2024    145
dtype: int64
<class 'pandas.core.series.Series'>
```

In [148…
```
print(df)
```

```
   miasto   rok  sprzedaz
0    łódź  2023       120
1    łódź  2024       145
2  Kraków  2023       200
3  Kraków  2024       230
```

In [149…
```
# Konwersja płaskiej ramki na hierarchiczną - set_index:
df_hier = df.set_index(["miasto", "rok"])
print(df_hier)
print(df_hier.index)
```

```
             sprzedaz
miasto rok
łódź   2023       120
       2024       145
Kraków 2023       200
       2024       230
MultiIndex([(  'łódź', 2023),
            (  'łódź', 2024),
            ('Kraków', 2023),
            ('Kraków', 2024)],
           names=['miasto', 'rok'])
```

In [150…
```
# I z powrotem - reset_index:
df2 = df_hier.reset_index()
print(df2)
```

```
   miasto   rok  sprzedaz
0    Łódź  2023       120
1    Łódź  2024       145
2  Kraków  2023       200
3  Kraków  2024       230
```

Tabele przestawne (ang. *Pivot Table*)

In [151…

```
# Przekształcamy płaską ramkę w tabelę przestawną
df2_my_pivot = df.set_index(["miasto", "rok"])["sprzedaz"].unstack()
print(df2_my_pivot)
```

```
rok     2023  2024
miasto
Kraków   200   230
Łódź     120   145
```

In [152…

```
df_pivot = pd.pivot_table(df,
                          index="miasto",
                          columns="rok",
                          values="sprzedaz")
print(df_pivot)
```

```
rok      2023   2024
miasto
Kraków  200.0  230.0
Łódź    120.0  145.0
```

In [153…

```
df_kwartaly = pd.DataFrame({
    "miasto":   ["Łódź"]*4 + ["Kraków"]*4,
    "rok":      [2023]*4 + [2023]*4,
    "kwartal":  ["Q1","Q2","Q3","Q4"] * 2,
    "sprzedaz": [30, 25, 35, 30, 50, 45, 55, 50]
})

df_kwartaly_sum = pd.pivot_table(df_kwartaly,
                                 index="miasto",
                                 columns="rok",
                                 values="sprzedaz",
                                 aggfunc="sum")
print(df_kwartaly_sum)
```

```
rok     2023
miasto
Kraków   200
Łódź     120
```

Agregacje po poziomie

In [154…

```
sprzedaz.index.names = ["miasto", "rok"]
sprzedaz.groupby(level="miasto").sum()
```

Out[154…

miasto	0
Kraków	430
Łódź	265

dtype: int64

In [155…

```
sprzedaz.groupby(level="rok").mean()
```

Out[155…

rok	0
2023	160.0
2024	187.5

dtype: float64

Agregacja w DataFrame z MultiIndex

In [156…

```
frame = pd.DataFrame({
    "sprzedaz": [120, 145, 200, 230],
    "koszty":   [80,  90,  130, 140]
}, index=[["Łódź","Łódź","Kraków","Kraków"],
          [2023,  2024,  2023,    2024]])
frame.index.names = ["miasto", "rok"]
frame.groupby(level="miasto").sum()
```

Out[156…

miasto	sprzedaz	koszty
Kraków	430	270
Łódź	265	170

In [157…

```
frame.groupby(level="miasto").agg(["sum", "mean"])
```

Out[157…

	sprzedaz		koszty	
miasto	sum	mean	sum	mean
Kraków	430	215.0	270	135.0
Łódź	265	132.5	170	85.0

In [158…

```
frame["sprzedaz"].groupby(level="miasto").agg(
    total="sum", srednia="mean", rozstep=lambda x: x.max()-x.min()
    )
```

Out[158…

miasto	total	srednia	rozstep
Kraków	430	215.0	30
Łódź	265	132.5	25

Zadania

Sieć "ModaŁódź" ma sklepy w trzech miastach. Każdy sklep sprzedaje trzy kategorie produktów. Dane obejmują 4 kwartały 2023 i 2024 roku.

In [159…

```
print(df.columns)
print(df.head())
```

```
Index(['miasto', 'rok', 'sprzedaz'], dtype='object')
   miasto   rok  sprzedaz
0    Łódź  2023       120
1    Łódź  2024       145
2  Kraków  2023       200
3  Kraków  2024       230
```

Zadanie 1 — Budowanie MultiIndex

a) Utwórz ramkę `df_hi` z hierarchicznym indeksem (`miasto` , `kategoria` , `rok` , `kwartal`). Posortuj indeks.

b) Wyświetl liczbę poziomów indeksu i ich nazwy.

c) Ile unikalnych kombinacji indeksu istnieje? Użyj `.index` do odpowiedzi.

In [160…

```
import pandas as pd

df = pd.read_csv("sklepy_moda.csv", index_col=0)

df_hi = df.set_index(["miasto", "kategoria", "rok", "kwartal"]).sort_index()

print(df)
print(df_hi.index.nlevels)
print(df_hi.index.names)
print(len(df_hi.index.unique()))
```

```
     miasto  kategoria   rok kwartal  przychod_tys  koszt_tys  sztuki
0      Łódź     Damska  2023      Q1          50.5       35.2     615
1      Łódź     Damska  2023      Q2          59.3       34.5     539
2      Łódź     Damska  2023      Q3          61.9       34.3     915
3      Łódź     Damska  2023      Q4          81.8       58.6     775
4      Łódź     Damska  2024      Q1          49.5       30.2     577
..      ...        ...   ...     ...           ...        ...     ...
67  Wrocław  Dziecięca  2023      Q4          50.5       36.8     627
68  Wrocław  Dziecięca  2024      Q1          36.4       24.2     416
69  Wrocław  Dziecięca  2024      Q2          52.6       31.0     686
70  Wrocław  Dziecięca  2024      Q3          35.5       26.2     521
71  Wrocław  Dziecięca  2024      Q4          61.4       45.0     650

[72 rows x 7 columns]
4
['miasto', 'kategoria', 'rok', 'kwartal']
72
```

Zadanie 2 — Selekcje na MultiIndex

a) Wybierz wszystkie dane dla Łodzi.

b) Wybierz dane dla Łodzi, kategorii Damska.

c) Wybierz dane dla wszystkich miast, ale tylko rok 2024. (Wskazówka: użyj metodę `xs()`)

d) Wybierz Q4 z obu lat, ale tylko dla Krakowa i kategorii Męska.

In [161…

```
dane_lodz = df_hi.loc["Łódź"]

dane_lodz_damska = df_hi.loc[("Łódź", "Damska")]

dane_2024 = df_hi.xs(2024, level="rok")

dane_krakow_meska_q4 = df_hi.loc[("Kraków", "Męska", slice(None), "Q4"), :]

print(dane_lodz)
print("\n--- ### ---\n")
print(dane_lodz_damska)
print("\n--- ### ---\n")
print(dane_2024)
print("\n--- ### ---\n")
print(dane_krakow_meska_q4)
```

```
                        przychod_tys  koszt_tys  sztuki
kategoria rok  kwartal
Damska    2023 Q1               50.5       35.2     615
               Q2               59.3       34.5     539
               Q3               61.9       34.3     915
               Q4               81.8       58.6     775
          2024 Q1               49.5       30.2     577
               Q2               62.5       39.8     627
               Q3               53.3       32.4     563
               Q4               85.8       55.0    1157
Dziecięca 2023 Q1               20.6       14.7     216
               Q2               31.4       19.0     370
               Q3               33.1       18.7     493
               Q4               43.6       30.7     409
          2024 Q1               35.4       24.9     301
               Q2               42.5       26.4     374
               Q3               35.7       22.0     301
               Q4               54.0       33.1     554
Męska     2023 Q1               40.3       26.9     335
               Q2               42.9       28.8     394
               Q3               42.0       28.8     465
               Q4               63.5       36.5     728
          2024 Q1               47.3       29.0     550
               Q2               47.9       31.6     445
               Q3               52.3       29.7     490
               Q4               71.1       39.7     730

--- ### ---

              przychod_tys  koszt_tys  sztuki
rok  kwartal
2023 Q1               50.5       35.2     615
     Q2               59.3       34.5     539
     Q3               61.9       34.3     915
     Q4               81.8       58.6     775
2024 Q1               49.5       30.2     577
     Q2               62.5       39.8     627
     Q3               53.3       32.4     563
     Q4               85.8       55.0    1157

--- ### ---

                          przychod_tys  koszt_tys  sztuki
miasto  kategoria kwartal
Kraków  Damska    Q1               73.0       44.7     843
                  Q2               89.2       65.3     869
                  Q3               89.0       53.0     759
                  Q4              115.3       70.1    1052
        Dziecięca Q1               48.1       28.7     629
                  Q2               56.7       35.4     704
                  Q3               50.6       28.7     700
                  Q4               79.7       48.9     741
        Męska     Q1               60.5       38.3     578
                  Q2               74.4       42.7     771
                  Q3               67.2       43.9     868
                  Q4              100.3       62.5    1484
Wrocław Damska    Q1               58.3       42.8     824
                  Q2               76.1       45.8     960
                  Q3               71.1       46.6     689
                  Q4              107.6       61.2    1536
```

```
        Dziecięca Q1              36.4       24.2     416
                  Q2              52.6       31.0     686
                  Q3              35.5       26.2     521
                  Q4              61.4       45.0     650
        Męska     Q1              53.3       29.4     486
                  Q2              61.2       40.4     786
                  Q3              52.4       36.3     506
                  Q4              87.5       53.8    1157
Łódź    Damska    Q1              49.5       30.2     577
                  Q2              62.5       39.8     627
                  Q3              53.3       32.4     563
                  Q4              85.8       55.0    1157
        Dziecięca Q1              35.4       24.9     301
                  Q2              42.5       26.4     374
                  Q3              35.7       22.0     301
                  Q4              54.0       33.1     554
        Męska     Q1              47.3       29.0     550
                  Q2              47.9       31.6     445
                  Q3              52.3       29.7     490
                  Q4              71.1       39.7     730

--- ### ---

                              przychod_tys  koszt_tys  sztuki
miasto kategoria rok  kwartal
Kraków Męska     2023 Q4              89.4       63.8    1253
                 2024 Q4             100.3       62.5    1484
```

Zadanie 3 — pivot_table: roczne podsumowanie

a) Utwórz tabelę przestawną `tab_miasta`, która pokaże **sumę przychodów** w wierszach per miasto, w kolumnach per rok.

b) Dodaj do tabeli kolumnę `zmiana_proc` — procentową zmianę przychodu między 2023 a 2024. Które miasto rosło najszybciej?

c) Utwórz drugą tabelę przestawną, w której wiersze to (miasto, kategoria), kolumny to rok, wartości to *średni przychód kwartalny*. Która kombinacja (miasto, kategoria) ma najwyższy średni przychód w 2024?

In [162…

```
tab_miasta = pd.pivot_table(df_hi.reset_index(), index="miasto", columns="rok",

tab_miasta["zmiana_proc"] = ((tab_miasta[2024] - tab_miasta[2023]) / tab_miasta[
print(tab_miasta["zmiana_proc"].idxmax())

tab_kategorie = pd.pivot_table(df_hi.reset_index(), index=["miasto", "kategoria"
print(tab_kategorie[2024].idxmax())
```

```
Łódź
('Kraków', 'Damska')
```

Zadanie 4 — Agregacja po poziomach

a) Na `df_hi` oblicz sumę przychodów per miasto (agreguj po poziomie `"miasto"`).

b) Oblicz **średni przychód kwartalny per kategoria** (agreguj po poziomie `"kategoria"`).

c) Użyj `.agg(["sum", "mean", "max"])` na kolumnie `przychod_tys` pogrupowanej po `(miasto, rok)`. Który wiersz ma najwyższą wartość `max`?

In [163…

```
suma_per_miasto = df_hi.groupby(level="miasto")["przychod_tys"].sum()

srednia_per_kategoria = df_hi.groupby(level="kategoria")["przychod_tys"].mean()

agg_miasto_rok = df_hi.groupby(level=["miasto", "rok"])["przychod_tys"].agg(["su
print(agg_miasto_rok["max"].idxmax())
```

```
('Kraków', np.int64(2023))
```

Zadanie 5 — Marża i ranking

a) W `df_hi` dodaj kolumnę `marza_tys = przychod_tys - koszt_tys`.

b) Utwórz tabelę przestawną: *wiersze* = *miasto*, *kolumny* = *kategoria*, *wartości* = *suma marży*. Które miasto jest najbardziej zyskowne? Która kategoria generuje najwyższą marżę?

In [164…

```
df_hi["marza_tys"] = df_hi["przychod_tys"] - df_hi["koszt_tys"]

tab_marza = pd.pivot_table(df_hi.reset_index(), index="miasto", columns="kategor

print(tab_marza.sum(axis=1).idxmax())

print(tab_marza.sum(axis=0).idxmax())
```

```
Kraków
Damska
```

In [165…

```
!jupyter nbconvert --to html "LAB5-MK-przetwarzanie_danych.ipynb"
```

```
[NbConvertApp] Converting notebook LAB5-MK-przetwarzanie_danych.ipynb to html
[NbConvertApp] Writing 383678 bytes to LAB5-MK-przetwarzanie_danych.html
```